

The first subtask can be solved with a simple *brute force* algorithm. For every possible choice of strings, we can calculate the size of the components resulting from their cutting, using breadth-first search or depth-first search. The complexity of this solution is $\mathcal{O}(n^3)$.

The second subtask can be solved by a smarter implementation of the previously described algorithm. Fix the first rope we will cut. We got two components. Let's look at one of them and root it in an arbitrary pepper. Using the depth-first search algorithm, we can efficiently calculate the size of subtree of each pepper, i.e. the sizes of the component in which the pepper will be located if we cut the string between it and its parent in the tree. The complexity of this solution is $\mathcal{O}(n^2)$.

We will also describe a different solution for the second subtask, which we will then optimize for the third subtask. Root the given tree in any pepper. Choosing two strings to cut can be viewed as choosing two peppers for which we will cut the strings connecting them to their parents.

For a pepper x , mark the size of its subtree by $S(x)$. We have two cases:

1. The pepper y is an ancestor of pepper x , i.e. pepper y is on the path from x to the root. The sizes of the components are then $S(x)$, $S(y) - S(x)$ and $n - S(y)$. (The case where x is an ancestor of y is similar.)
2. The pepper x is not an ancestor of y , nor y is an ancestor of x . The sizes of the components are then $S(x)$, $S(y)$ and $n - S(x) - S(y)$.

This approach can be implemented directly in the complexity of $\mathcal{O}(n^2)$.

To solve the third subtask, we will optimize this approach. During depth-first search, we will maintain two sets of already processed peppers: peppers that are ancestors of current peppers (set A) and those that are not (set B). When we enter some pepper, we add it to the set A . When we go out, we throw out the current pepper from the set A , and add it to the set B .

Consider the first case for the current pepper x . We are looking for y from the set A for which the numbers $S(x)$, $S(y) - S(x)$ and $n - S(y)$ are "as equal as possible". That is achieved for y for which the value $\left|S(y) - \frac{n+S(x)}{2}\right|$ is minimal (proof left as exercise). Subtree sizes can be kept in a C++ `std::set`, and we can use `set::lower_bound()` to find the values that are closest to $\frac{n+S(x)}{2}$ on both sides. The second case is solved analogously, using the set B .

The complexity of this solution is $\mathcal{O}(n \log n)$. For implementation details, see the official solution.